Arquitetura de Sistema Next.js para Gestão de Vinícolas

Autor: Manus AI**Data:** Abril de 2026**Versão:** 1.0**Escopo:** Arquitetura completa de SaaS multi-tenant para gestão integrada de vinícolas

---

Sumário Executivo

Este documento descreve a arquitetura de um **Sistema de Gestão Integrado para Vinícolas (SGIV)** desenvolvido em Next.js com suporte a multi-tenancy, autenticação robusta, e cinco módulos principais: Gestão de Produtos, Enoturismo, Vendas, Equipe e Analytics. O sistema permite que cada vinícola (empresa) tenha seu próprio banco de dados isolado, com controle granular de permissões por papel (Owner, Manager, Employee).

---

1. Visão Geral da Arquitetura

1.1 Princípios Arquiteturais

Multi-Tenancy: Cada empresa (vinícola) possui banco de dados separado, garantindo isolamento de dados e segurança.

Autenticação Robusta: Suporte a email/senha e OAuth Google, com sistema de roles e permissões granulares.

Modularidade: Cinco módulos independentes mas integrados (Produtos, Enoturismo, Vendas, Equipe, Analytics).

Escalabilidade: Arquitetura preparada para crescimento de usuários e dados sem degradação de performance.

User Experience: Interface intuitiva para donos, gerentes e funcionários com dashboards personalizados por papel.

1.2 Stack Tecnológico

Camada	Tecnologia	Justificativa
Frontend	Next.js 14+ (React 19)	SSR, SSG, API Routes, performance otimizada

UI/Styling	Tailwind CSS 4 + shadcn/ui	Design system moderno, acessível, responsivo
Autenticação	NextAuth.js v5	OAuth, JWT, sessões, multi-provider
Banco de Dados	PostgreSQL + Prisma ORM	Multi-tenancy, migrations, type-safe queries
API	Next.js API Routes	Serverless, integrado ao framework
Validação	Zod	Type-safe validation, schemas
Estado Global	React Context + Zustand	Gerenciamento de estado leve e eficiente
Relatórios	Recharts + PDF Export	Gráficos interativos, exportação de relatórios
Deployment	Vercel / Railway	Escalabilidade, CI/CD automático

---

2. Estrutura de Pastas e Arquivos

Plain Text

```
vinicola-system/
├── app/                                # App Router (Next.js 14+)
│   ├── (auth)/                         # Grupo de rotas de autenticação
│   │   ├── login/
│   │   │   └── page.tsx
│   │   ├── register/
│   │   │   └── page.tsx
│   │   ├── oauth/
│   │   │   └── callback/page.tsx
│   │   └── forgot-password/
│   │       └── page.tsx
│   │
│   ├── (dashboard)/                    # Grupo de rotas autenticadas
│   │   ├── layout.tsx                  # Layout principal com sidebar
│   │   ├── dashboard/                  # Home/Dashboard
│   │   │   └── page.tsx
│   │   │
│   │   ├── produtos/                   # Módulo de Produtos
```

```
│   │   │   ├── page.tsx                    # Lista de produtos
│   │   │   ├── [id]/
│   │   │   │   ├── page.tsx                # Detalhes do produto
│   │   │   │   └── editar/page.tsx
│   │   │   ├── novo/page.tsx               # Criar novo produto
│   │   │   └── categorias/page.tsx         # Gerenciar categorias
│   │   │
│   │   ├── enoturismo/                     # Módulo de Enoturismo
│   │   │   ├── page.tsx                    # Dashboard enoturismo
│   │   │   ├── visitas/
│   │   │   │   ├── page.tsx
│   │   │   │   ├── [id]/page.tsx
│   │   │   │   └── nova/page.tsx
│   │   │   ├── degustacoes/
│   │   │   │   ├── page.tsx
│   │   │   │   └── nova/page.tsx
│   │   │   ├── eventos/
│   │   │   │   ├── page.tsx
│   │   │   │   └── novo/page.tsx
│   │   │   ├── hospedagem/
│   │   │   │   ├── page.tsx
│   │   │   │   └── novo/page.tsx
│   │   │   └── cursos/
│   │   │       ├── page.tsx
│   │   │       └── novo/page.tsx
│   │   │
│   │   ├── vendas/                         # Módulo de Vendas
│   │   │   ├── page.tsx                    # Dashboard vendas
│   │   │   ├── pedidos/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/page.tsx
│   │   │   ├── clientes/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/page.tsx
│   │   │   ├── distribuicao/
│   │   │   │   ├── page.tsx                # Gestão de distribuidores
│   │   │   │   └── novo/page.tsx
│   │   │   ├── clube-assinatura/
│   │   │   │   ├── page.tsx
│   │   │   │   └── novo/page.tsx
│   │   │   └── ecommerce/
│   │   │       ├── page.tsx
│   │   │       └── configuracoes/page.tsx
│   │   │
│   │   ├── equipe/                         # Módulo de Equipe
│   │   │   ├── page.tsx                    # Lista de funcionários
│   │   │   ├── [id]/page.tsx               # Perfil do funcionário
│   │   │   ├── novo/page.tsx               # Adicionar funcionário
```

```
│   │   │   ├── papeis/page.tsx             # Gerenciar papéis
│   │   │   └── permissoes/page.tsx         # Gerenciar permissões
│   │   │
│   │   ├── analytics/                      # Módulo de Analytics
│   │   │   ├── page.tsx                    # Dashboard principal
│   │   │   ├── produtos/page.tsx           # Analytics de produtos
│   │   │   ├── vendas/page.tsx             # Analytics de vendas
│   │   │   ├── enoturismo/page.tsx         # Analytics de enoturismo
│   │   │   ├── equipe/page.tsx             # Analytics de equipe
│   │   │   └── relatorios/page.tsx         # Geração de relatórios
│   │   │
│   │   ├── configuracoes/                  # Configurações da empresa
│   │   │   ├── page.tsx                    # Home de configurações
│   │   │   ├── empresa/page.tsx            # Dados da empresa
│   │   │   ├── integracao/page.tsx         # Integrações (Stripe, etc)
│   │   │   ├── notificacoes/page.tsx       # Preferências de notificação
│   │   │   └── seguranca/page.tsx          # Segurança e 2FA
│   │   │
│   │   └── perfil/page.tsx                 # Perfil do usuário
│   │
│   ├── api/                                # API Routes
│   │   ├── auth/
│   │   │   ├── [...nextauth]/route.ts      # NextAuth handler
│   │   │   ├── register/route.ts
│   │   │   └── logout/route.ts
│   │   │
│   │   ├── produtos/
│   │   │   ├── route.ts                    # GET/POST produtos
│   │   │   ├── [id]/route.ts               # GET/PUT/DELETE produto
│   │   │   └── categorias/route.ts
│   │   │
│   │   ├── enoturismo/
│   │   │   ├── visitas/route.ts
│   │   │   ├── degustacoes/route.ts
│   │   │   ├── eventos/route.ts
│   │   │   ├── hospedagem/route.ts
│   │   │   └── cursos/route.ts
│   │   │
│   │   ├── vendas/
│   │   │   ├── pedidos/route.ts
│   │   │   ├── clientes/route.ts
│   │   │   ├── distribuicao/route.ts
│   │   │   ├── clube-assinatura/route.ts
│   │   │   └── ecommerce/route.ts
│   │   │
│   │   ├── equipe/
│   │   │   ├── funcionarios/route.ts
│   │   │   ├── papeis/route.ts
```

```
│   │   │   └── permissoes/route.ts
│   │   │
│   │   ├── analytics/
│   │   │   ├── dashboard/route.ts
│   │   │   ├── relatorios/route.ts
│   │   │   └── export/route.ts
│   │   │
│   │   ├── configuracoes/
│   │   │   ├── empresa/route.ts
│   │   │   └── integracao/route.ts
│   │   │
│   │   └── webhook/
│   │       ├── stripe/route.ts
│   │       └── pagamento/route.ts
│   │
│   ├── layout.tsx                          # Layout raiz
│   └── page.tsx                            # Landing page
│
├── components/                             # Componentes reutilizáveis
│   ├── auth/
│   │   ├── LoginForm.tsx
│   │   ├── RegisterForm.tsx
│   │   ├── OAuthButton.tsx
│   │   └── ProtectedRoute.tsx
│   │
│   ├── layout/
│   │   ├── Sidebar.tsx
│   │   ├── Header.tsx
│   │   ├── Footer.tsx
│   │   └── Navigation.tsx
│   │
│   ├── produtos/
│   │   ├── ProdutoCard.tsx
│   │   ├── ProdutoForm.tsx
│   │   ├── ProdutoList.tsx
│   │   ├── CategoriaSelect.tsx
│   │   └── ProdutoModal.tsx
│   │
│   ├── enoturismo/
│   │   ├── VisitaCard.tsx
│   │   ├── VisitaForm.tsx
│   │   ├── DegustacaoCard.tsx
│   │   ├── EventoForm.tsx
│   │   ├── HospedagemCard.tsx
│   │   └── CursoForm.tsx
│   │
│   ├── vendas/
│   │   ├── PedidoCard.tsx
```

```
│   │   ├── PedidoForm.tsx
│   │   ├── ClienteCard.tsx
│   │   ├── DistribuidorForm.tsx
│   │   ├── ClubeAssinaturaForm.tsx
│   │   └── CarrinhoCompras.tsx
│   │
│   ├── equipe/
│   │   ├── FuncionarioCard.tsx
│   │   ├── FuncionarioForm.tsx
│   │   ├── PapelForm.tsx
│   │   ├── PermissaoMatrix.tsx
│   │   └── FuncionarioList.tsx
│   │
│   ├── analytics/
│   │   ├── DashboardCard.tsx
│   │   ├── GraficoVendas.tsx
│   │   ├── GraficoEnoturismo.tsx
│   │   ├── GraficoProdutos.tsx
│   │   ├── RelatorioExport.tsx
│   │   └── MetricasCard.tsx
│   │
│   ├── ui/                                 # shadcn/ui components
│   │   ├── button.tsx
│   │   ├── card.tsx
│   │   ├── form.tsx
│   │   ├── input.tsx
│   │   ├── select.tsx
│   │   ├── table.tsx
│   │   ├── modal.tsx
│   │   ├── tabs.tsx
│   │   ├── dropdown.tsx
│   │   └── ... (outros componentes shadcn)
│   │
│   └── common/
│       ├── LoadingSpinner.tsx
│       ├── ErrorBoundary.tsx
│       ├── Toast.tsx
│       └── ConfirmDialog.tsx
│
├── lib/                                    # Utilitários e helpers
│   ├── auth/
│   │   ├── authOptions.ts                  # Configuração NextAuth
│   │   ├── middleware.ts                   # Middleware de autenticação
│   │   ├── permissions.ts                  # Verificação de permissões
│   │   └── roles.ts                        # Definição de papéis
│   │
│   ├── db/
│   │   ├── prisma.ts                       # Cliente Prisma
```

```
│   │   ├── tenant.ts                       # Isolamento multi-tenant
│   │   └── seed.ts                         # Seed de dados iniciais
│   │
│   ├── api/
│   │   ├── client.ts                       # Cliente HTTP para chamadas API
│   │   ├── handlers.ts                     # Handlers de erro
│   │   └── validation.ts                   # Validação de requisições
│   │
│   ├── utils/
│   │   ├── formatters.ts                   # Formatação de dados
│   │   ├── validators.ts                   # Validadores customizados
│   │   ├── constants.ts                    # Constantes da aplicação
│   │   └── helpers.ts                      # Funções auxiliares
│   │
│   ├── hooks/
│   │   ├── useAuth.ts                      # Hook de autenticação
│   │   ├── useTenant.ts                    # Hook de tenant
│   │   ├── usePermission.ts                # Hook de permissões
│   │   ├── useFetch.ts                     # Hook de fetch com cache
│   │   └── useForm.ts                      # Hook de formulário
│   │
│   └── types/
│       ├── index.ts                        # Tipos globais
│       ├── auth.ts                         # Tipos de autenticação
│       ├── produtos.ts                     # Tipos de produtos
│       ├── enoturismo.ts                   # Tipos de enoturismo
│       ├── vendas.ts                       # Tipos de vendas
│       ├── equipe.ts                       # Tipos de equipe
│       └── analytics.ts                    # Tipos de analytics
│
├── contexts/                               # React Contexts
│   ├── AuthContext.tsx
│   ├── TenantContext.tsx
│   ├── PermissionContext.tsx
│   └── ThemeContext.tsx
│
├── stores/                                 # Zustand stores
│   ├── authStore.ts
│   ├── tenantStore.ts
│   ├── produtosStore.ts
│   ├── vendasStore.ts
│   └── uiStore.ts
│
├── prisma/                                 # Prisma ORM
│   ├── schema.prisma                       # Schema do banco
│   └── migrations/                         # Histórico de migrations
│
├── public/                                 # Arquivos estáticos
```

```
│   ├── images/
│   ├── icons/
│   └── logo.svg
│
├── styles/                                # Estilos globais
│   └── globals.css
│
├── .env.local                             # Variáveis de ambiente
├── .env.example
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
├── package.json
└── README.md
```

---

3. Autenticação e Autorização

3.1 Sistema de Autenticação

3.1.1 NextAuth.js Configuration (`lib/auth/authOptions.ts`)

TypeScript

```
// Suporte a email/senha e OAuth Google
// JWT com informações de tenant e papéis
// Callbacks para validação e autorização
// Suporte a 2FA opcional
```

Fluxo de Autenticação:

1. Usuário acessa `/login`
2. Escolhe: Email/Senha ou Google OAuth
3. Se Email/Senha: validação contra banco de dados
4. Se Google: redirecionamento para Google, callback com criação de usuário
5. Geração de JWT com tenant_id e roles
6. Redirect para dashboard

3.1.2 Estrutura de Usuário e Tenant

TypeScript

```
// Usuário
interface User {
  id: string
  email: string
  name: string
  avatar?: string
  tenantId: string           // Isolamento multi-tenant
  roles: Role[]              // Array de papéis
  permissions: Permission[]  // Array de permissões
  createdAt: Date
  updatedAt: Date
}

// Tenant (Empresa/Vinícola)
interface Tenant {
  id: string
  name: string               // Nome da vinícola
  slug: string               // URL-friendly identifier
  databaseUrl: string        // URL do banco isolado
  owner: User
  users: User[]
  subscription: Subscription
  createdAt: Date
  updatedAt: Date
}

// Papel (Role)
interface Role {
  id: string
  name: string               // "Owner", "Manager", "Employee"
  description: string
  permissions: Permission[]
  tenantId: string
  createdAt: Date
}

// Permissão
interface Permission {
  id: string
  name: string               // "create_product", "view_sales", etc
  description: string
  module: string             // "produtos", "vendas", "enoturismo", etc
  action: string             // "create", "read", "update", "delete"
  roles: Role[]
  tenantId: string
}
```

3.2 Sistema de Papéis e Permissões

3.2.1 Papéis Predefinidos

Papel	Descrição	Permissões
Owner	Proprietário da vinícola	Todas as permissões
Manager	Gerente/Administrador	Todas exceto: deletar empresa, gerenciar assinatura
Employee	Funcionário	Apenas leitura + ações específicas de seu departamento

3.2.2 Matriz de Permissões

TypeScript

```typescript
// Exemplo de permissões por módulo
const PERMISSIONS = {
  PRODUTOS: {
    CREATE: "produtos:create",
    READ: "produtos:read",
    UPDATE: "produtos:update",
    DELETE: "produtos:delete",
    EXPORT: "produtos:export"
  },
  VENDAS: {
    CREATE_PEDIDO: "vendas:create_pedido",
    READ_PEDIDOS: "vendas:read_pedidos",
    UPDATE_PEDIDO: "vendas:update_pedido",
    DELETE_PEDIDO: "vendas:delete_pedido",
    VIEW_RELATORIOS: "vendas:view_relatorios"
  },
  ENOTURISMO: {
    CREATE_VISITA: "enoturismo:create_visita",
    READ_VISITAS: "enoturismo:read_visitas",
    UPDATE_VISITA: "enoturismo:update_visita",
    DELETE_VISITA: "enoturismo:delete_visita"
  },
  EQUIPE: {
    CREATE_FUNCIONARIO: "equipe:create_funcionario",
    READ_FUNCIONARIOS: "equipe:read_funcionarios",
    UPDATE_FUNCIONARIO: "equipe:update_funcionario",
```

```
    DELETE_FUNCIONARIO: "equipe:delete_funcionario",
    MANAGE_ROLES: "equipe:manage_roles"
  },
  ANALYTICS: {
    VIEW_DASHBOARD: "analytics:view_dashboard",
    VIEW_RELATORIOS: "analytics:view_relatorios",
    EXPORT_DADOS: "analytics:export_dados"
  }
}
```

3.3 Middleware de Autenticação

TypeScript

```
// middleware.ts
// Verifica token JWT
// Valida tenant_id
// Redireciona não autenticados para login
// Aplica rate limiting
```

---

4. Multi-Tenancy e Isolamento de Dados

4.1 Estratégia de Multi-Tenancy

Modelo: Database-per-Tenant (cada empresa tem seu próprio banco PostgreSQL)

Vantagens:

- Isolamento completo de dados
- Segurança máxima
- Escalabilidade independente por tenant
- Backup e restore granular

Desvantagens:

- Complexidade operacional aumentada
- Custo de infraestrutura maior
- Gerenciamento de múltiplas conexões

4.2 Implementação de Multi-Tenancy

4.2.1 Estrutura de Banco de Dados

TypeScript

```
// Banco "master" (compartilhado)
// Armazena: Tenants, Usuários, Subscriptions, Integrações

// Banco por Tenant (isolado)
// Armazena: Produtos, Vendas, Enoturismo, Equipe, Analytics

// Conexão dinâmica baseada em tenant_id do JWT
```

4.2.2 Prisma com Multi-Tenancy

TypeScript

```
// lib/db/tenant.ts
// Função para obter cliente Prisma correto baseado em tenant_id
// Caching de conexões para performance

export async function getTenantPrisma(tenantId: string) {
  // 1. Buscar tenant no banco master
  // 2. Obter databaseUrl
  // 3. Retornar cliente Prisma com essa URL
  // 4. Cachear conexão
}

// Uso em API routes:
const prisma = await getTenantPrisma(tenantId)
const produtos = await prisma.produto.findMany()
```

4.2.3 Middleware de Tenant

TypeScript

```
// lib/auth/middleware.ts
// Middleware que:
// 1. Extrai tenant_id do JWT
// 2. Valida se usuário pertence ao tenant
// 3. Injeta tenant_id em todas as requisições
// 4. Garante isolamento de dados
```

---

5. Schema Prisma (Banco de Dados)

5.1 Banco Master (Compartilhado)

Plain Text

```
// Master database schema
model Tenant {
  id            String    @id @default(cuid())
  name          String
  slug          String    @unique
  databaseUrl   String    // URL do banco isolado
  logo          String?
  website       String?
  phone         String?
  address       String?
  city          String?
  state         String?
  zipCode       String?
  owner         User      @relation("TenantOwner", fields: [ownerId], reference
  ownerId       String
  users         User[]    @relation("TenantUsers")
  subscription  Subscription?
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
}

model User {
  id            String    @id @default(cuid())
  email         String    @unique
  name          String
  password      String?   // Hash de senha
  avatar        String?
  provider      String?   // "credentials", "google"
  providerId    String?   // ID do OAuth
  tenants       Tenant[]  @relation("TenantUsers")
  ownedTenants  Tenant[]  @relation("TenantOwner")
  roles         Role[]
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
}

model Subscription {
  id            String    @id @default(cuid())
  tenant        Tenant    @relation(fields: [tenantId], references: [id])
  tenantId      String    @unique
  plan          String    // "starter", "professional", "enterprise"
  status        String    // "active", "canceled", "past_due"
  stripeId      String?
  currentPeriodStart DateTime
```

```
  currentPeriodEnd DateTime
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
}

model Role {
  id            String    @id @default(cuid())
  name          String
  description   String?
  permissions   Permission[]
  users         User[]
  createdAt     DateTime  @default(now())
}

model Permission {
  id            String    @id @default(cuid())
  name          String    // "create_product", "view_sales"
  description   String?
  module        String    // "produtos", "vendas", "enoturismo"
  action        String    // "create", "read", "update", "delete"
  roles         Role[]
}
```

5.2 Banco Tenant (Isolado por Empresa)

Plain Text

```
// Tenant-specific database schema

// ============ MÓDULO: PRODUTOS ============
model Categoria {
  id            String    @id @default(cuid())
  name          String
  description   String?
  produtos      Produto[]
  createdAt     DateTime  @default(now())
}

model Produto {
  id            String    @id @default(cuid())
  name          String
  descricao     String?
  categoria     Categoria @relation(fields: [categoriaId], references: [id])
  categoriaId   String
  tipo          String    // "tinto", "branco", "rosé", "espumante", "suco", et
  preco         Decimal   @db.Decimal(10, 2)
  precoDistribuidor Decimal? @db.Decimal(10, 2)
```

```
  estoque       Int
  safra         Int?
  uva           String?
  alcool        Decimal?  @db.Decimal(3, 1)
  volume        String?   // "750ml", "1.5L", "3L"
  imagem        String?
  ativo         Boolean   @default(true)
  criadoEm      DateTime  @default(now())
  atualizadoEm  DateTime  @updatedAt
}

// ============ MÓDULO: ENOTURISMO ============
model Visita {
  id            String    @id @default(cuid())
  titulo        String
  descricao     String?
  duracao       Int       // em minutos
  preco         Decimal   @db.Decimal(10, 2)
  capacidade    Int
  tipo          String    // "basica", "completa", "premium", "colheita"
  status        String    // "ativa", "inativa"
  agendamentos  Agendamento[]
  criadoEm      DateTime  @default(now())
}

model Degustacao {
  id            String    @id @default(cuid())
  titulo        String
  descricao     String?
  duracao       Int
  preco         Decimal   @db.Decimal(10, 2)
  tipo          String    // "orientada", "cegas", "harmonizada"
  vinhos        Produto[]
  agendamentos  Agendamento[]
  criadoEm      DateTime  @default(now())
}

model Evento {
  id            String    @id @default(cuid())
  titulo        String
  descricao     String?
  dataInicio    DateTime
  dataFim       DateTime
  preco         Decimal?  @db.Decimal(10, 2)
  capacidade    Int?
  tipo          String    // "vindima", "festa_uva", "jantar_tematico"
  status        String    // "planejado", "confirmado", "realizado", "cancelado
  agendamentos  Agendamento[]
```

```
  criadoEm      DateTime  @default(now())
}

model Hospedagem {
  id            String    @id @default(cuid())
  titulo        String
  descricao     String?
  tipo          String    // "wine_house", "glamping", "pousada"
  preco         Decimal   @db.Decimal(10, 2)
  capacidade    Int
  amenidades    String[]
  imagens       String[]
  status        String    // "disponivel", "indisponivel"
  reservas      Reserva[]
  criadoEm      DateTime  @default(now())
}

model Curso {
  id            String    @id @default(cuid())
  titulo        String
  descricao     String?
  duracao       Int       // em horas
  preco         Decimal   @db.Decimal(10, 2)
  capacidade    Int
  tipo          String    // "vinificacao", "degustacao", "sommelier"
  nivel         String    // "basico", "intermediario", "avancado"
  instrutor     String?
  agendamentos  Agendamento[]
  criadoEm      DateTime  @default(now())
}

model Agendamento {
  id            String    @id @default(cuid())
  cliente       Cliente   @relation(fields: [clienteId], references: [id])
  clienteId     String
  visita        Visita?   @relation(fields: [visitaId], references: [id])
  visitaId      String?
  degustacao    Degustacao? @relation(fields: [degustacaoId], references: [id])
  degustacaoId  String?
  evento        Evento?   @relation(fields: [eventoId], references: [id])
  eventoId      String?
  curso         Curso?    @relation(fields: [cursoId], references: [id])
  cursoId       String?
  dataAgendamento DateTime
  quantidadePessoas Int
  status        String    // "confirmado", "cancelado", "realizado"
  preco         Decimal   @db.Decimal(10, 2)
  criadoEm      DateTime  @default(now())
```

```
}

model Reserva {
  id            String    @id @default(cuid())
  hospedagem    Hospedagem @relation(fields: [hospedagemId], references: [id])
  hospedagemId  String
  cliente       Cliente   @relation(fields: [clienteId], references: [id])
  clienteId     String
  dataCheckin   DateTime
  dataCheckout  DateTime
  quantidadeHospedes Int
  preco         Decimal   @db.Decimal(10, 2)
  status        String    // "confirmada", "cancelada", "realizada"
  criadoEm      DateTime  @default(now())
}

// ============ MÓDULO: VENDAS ============
model Cliente {
  id            String    @id @default(cuid())
  nome          String
  email         String    @unique
  telefone      String?
  endereco      String?
  cidade        String?
  estado        String?
  cep           String?
  tipo          String    // "pessoa_fisica", "pessoa_juridica", "distribuidor"
  cpfCnpj       String?   @unique
  pedidos       Pedido[]
  agendamentos  Agendamento[]
  reservas      Reserva[]
  assinaturas   Assinatura[]
  criadoEm      DateTime  @default(now())
}

model Pedido {
  id            String    @id @default(cuid())
  cliente       Cliente   @relation(fields: [clienteId], references: [id])
  clienteId     String
  itens         ItemPedido[]
  status        String    // "pendente", "confirmado", "enviado", "entregue", "
  total         Decimal   @db.Decimal(10, 2)
  frete         Decimal?  @db.Decimal(10, 2)
  desconto      Decimal?  @db.Decimal(10, 2)
  metodo_pagamento String? // "pix", "cartao", "boleto", "transferencia"
  status_pagamento String? // "pendente", "confirmado", "falhou"
  canal         String    // "ecommerce", "distribuidor", "on_trade"
  criadoEm      DateTime  @default(now())
```

```
  atualizadoEm  DateTime  @updatedAt
}

model ItemPedido {
  id            String    @id @default(cuid())
  pedido        Pedido    @relation(fields: [pedidoId], references: [id])
  pedidoId      String
  produto       Produto   @relation(fields: [produtoId], references: [id])
  produtoId     String
  quantidade    Int
  preco_unitario Decimal  @db.Decimal(10, 2)
  subtotal      Decimal   @db.Decimal(10, 2)
}

model Distribuidor {
  id            String    @id @default(cuid())
  nome          String
  email         String    @unique
  telefone      String?
  endereco      String?
  cidade        String?
  estado        String?
  cnpj          String?   @unique
  tipo          String    // "distribuidor", "varejo", "on_trade"
  status        String    // "ativo", "inativo"
  comissao      Decimal?  @db.Decimal(5, 2) // percentual
  pedidos       Pedido[]
  criadoEm      DateTime  @default(now())
}

model Assinatura {
  id            String    @id @default(cuid())
  cliente       Cliente   @relation(fields: [clienteId], references: [id])
  clienteId     String
  plano         String    // "classic", "fresh", "premium", "selection"
  preco_mensal  Decimal   @db.Decimal(10, 2)
  frequencia    String    // "mensal", "bimestral", "trimestral"
  status        String    // "ativa", "pausada", "cancelada"
  proxima_entrega DateTime
  criadoEm      DateTime  @default(now())
  atualizadoEm  DateTime  @updatedAt
}

// ============ MÓDULO: EQUIPE ============
model Funcionario {
  id            String    @id @default(cuid())
  nome          String
  email         String    @unique
```

```
  telefone      String?
  cargo         String
  departamento  String    // "vendas", "enoturismo", "producao", "administrativ
  data_admissao DateTime
  status        String    // "ativo", "inativo", "afastado"
  salario       Decimal?  @db.Decimal(10, 2)
  roles         Role[]
  criadoEm      DateTime  @default(now())
}

model Role {
  id            String    @id @default(cuid())
  nome          String    // "Owner", "Manager", "Employee"
  descricao     String?
  permissoes    Permission[]
  funcionarios  Funcionario[]
}

model Permission {
  id            String    @id @default(cuid())
  nome          String
  descricao     String?
  modulo        String
  acao          String
  roles         Role[]
}

// ============ MÓDULO: ANALYTICS ============
model Metrica {
  id            String    @id @default(cuid())
  tipo          String    // "vendas", "enoturismo", "produtos", "equipe"
  data          DateTime
  valor         Decimal   @db.Decimal(15, 2)
  dimensao      String?   // ex: "produto_id", "categoria_id"
  criadoEm      DateTime  @default(now())
}

model Relatorio {
  id            String    @id @default(cuid())
  titulo        String
  tipo          String    // "vendas", "enoturismo", "produtos", "equipe"
  periodo_inicio DateTime
  periodo_fim   DateTime
  dados         Json      // Dados do relatório em JSON
  criado_por    String    // ID do funcionário
  criadoEm      DateTime  @default(now())
}
```

6. Módulos Principais

6.1 Módulo de Produtos

Funcionalidades:

- CRUD de produtos (vinhos, espumantes, derivados)
- Categorização de produtos
- Gestão de estoque
- Preços por segmento (varejo, distribuidor)
- Upload de imagens
- Filtros e busca avançada

Rotas API:

- `GET /api/produtos` — Listar produtos
- `POST /api/produtos` — Criar produto
- `GET /api/produtos/[id]` — Detalhes do produto
- `PUT /api/produtos/[id]` — Atualizar produto
- `DELETE /api/produtos/[id]` — Deletar produto
- `GET /api/produtos/categorias` — Listar categorias

Componentes:

- `ProdutoCard` — Card de exibição
- `ProdutoForm` — Formulário de criação/edição
- `ProdutoList` — Lista com filtros
- `CategoriaSelect` — Seletor de categoria

6.2 Módulo de Enoturismo

Funcionalidades:

- Gestão de visitas guiadas
- Agendamento de degustações
- Gestão de eventos (Vindima, Festa da Uva)
- Reserva de hospedagem
- Cursos e workshops

- Calendário de disponibilidade
- Notificações de agendamento

Rotas API:

- `GET /api/enoturismo/visitas` — Listar visitas
- `POST /api/enoturismo/visitas` — Criar visita
- `GET /api/enoturismo/agendamentos` — Listar agendamentos
- `POST /api/enoturismo/agendamentos` — Criar agendamento
- `GET /api/enoturismo/hospedagem` — Listar hospedagem
- `POST /api/enoturismo/reservas` — Criar reserva

Componentes:

- `VisitaCard` — Card de visita
- `DegustacaoForm` — Formulário de degustação
- `CalendarioDisponibilidade` — Calendário interativo
- `AgendamentoModal` — Modal de agendamento

6.3 Módulo de Vendas

Funcionalidades:

- Gestão de pedidos (e-commerce, distribuição, on-trade)
- Gestão de clientes
- Gestão de distribuidores
- Clube de assinatura
- Carrinho de compras
- Processamento de pagamentos (Stripe)
- Rastreamento de pedidos

Rotas API:

- `GET /api/vendas/pedidos` — Listar pedidos
- `POST /api/vendas/pedidos` — Criar pedido
- `GET /api/vendas/clientes` — Listar clientes
- `POST /api/vendas/clientes` — Criar cliente
- `GET /api/vendas/clube-assinatura` — Listar assinaturas

- `POST /api/vendas/clube-assinatura` — Criar assinatura

Componentes:

- `PedidoCard` — Card de pedido
- `CarrinhoCompras` — Carrinho de e-commerce
- `ClienteForm` — Formulário de cliente
- `ClubeAssinaturaForm` — Formulário de assinatura

6.4 Módulo de Equipe

Funcionalidades:

- Gestão de funcionários
- Atribuição de papéis
- Gestão de permissões granulares
- Histórico de atividades
- Controle de acesso por módulo

Rotas API:

- `GET /api/equipe/funcionarios` — Listar funcionários
- `POST /api/equipe/funcionarios` — Adicionar funcionário
- `PUT /api/equipe/funcionarios/[id]` — Atualizar funcionário
- `DELETE /api/equipe/funcionarios/[id]` — Deletar funcionário
- `GET /api/equipe/papeis` — Listar papéis
- `POST /api/equipe/permissoes` — Atualizar permissões

Componentes:

- `FuncionarioCard` — Card de funcionário
- `FuncionarioForm` — Formulário de funcionário
- `PermissaoMatrix` — Matriz de permissões
- `PapelForm` — Formulário de papel

6.5 Módulo de Analytics

Funcionalidades:

- Dashboard com KPIs principais

- Gráficos de vendas (receita, volume, tendências)
- Análise de enoturismo (visitantes, receita)
- Análise de produtos (mais vendidos, estoque)
- Análise de equipe (performance, produtividade)
- Exportação de relatórios (PDF, Excel)
- Filtros por período e dimensão

Rotas API:

- `GET /api/analytics/dashboard` — Dados do dashboard
- `GET /api/analytics/vendas` — Dados de vendas
- `GET /api/analytics/enoturismo` — Dados de enoturismo
- `GET /api/analytics/produtos` — Dados de produtos
- `POST /api/analytics/relatorios/export` — Exportar relatório

Componentes:

- `DashboardCard` — Card de métrica
- `GraficoVendas` — Gráfico de vendas (Recharts)
- `GraficoEnoturismo` — Gráfico de enoturismo
- `RelatorioExport` — Exportador de relatórios

---

7. Fluxos Principais

7.1 Fluxo de Autenticação

Plain Text

```
1. Usuário acessa /login
2. Escolhe: Email/Senha ou Google
3. Se Email/Senha:
   - Valida credenciais contra banco master
   - Gera JWT com tenant_id e roles
   - Redirect para /dashboard
4. Se Google:
   - Redirecionamento para Google
   - Callback em /api/auth/callback/google
   - Cria usuário se não existe
   - Gera JWT
   - Redirect para /dashboard
```

```
5. Middleware valida JWT em todas as rotas
6. Usuário acessa dashboard com acesso ao banco isolado
```

7.2 Fluxo de Criação de Produto

Plain Text

```
1. Owner/Manager acessa /produtos/novo
2. Preenche formulário com dados do produto
3. Seleciona categoria
4. Upload de imagem
5. Clica "Salvar"
6. Validação no cliente (Zod)
7. POST /api/produtos com dados
8. Middleware valida tenant_id
9. Prisma salva em banco isolado do tenant
10. Retorna produto criado
11. Redirect para /produtos/[id]
12. Toast de sucesso
```

7.3 Fluxo de Agendamento de Visita

Plain Text

```
1. Cliente acessa página de enoturismo
2. Visualiza visitas disponíveis
3. Clica em "Agendar"
4. Modal de agendamento abre
5. Seleciona data e quantidade de pessoas
6. Preenche dados de contato
7. Clica "Confirmar"
8. POST /api/enoturismo/agendamentos
9. Validação de disponibilidade
10. Cria agendamento no banco
11. Envia email de confirmação
12. Retorna confirmação ao cliente
```

7.4 Fluxo de Venda (E-commerce)

Plain Text

```
1. Cliente acessa /ecommerce
2. Navega por produtos
3. Adiciona produtos ao carrinho
```

```
4. Clica "Finalizar Compra"
5. Preenche dados de entrega
6. Seleciona método de pagamento
7. POST /api/vendas/pedidos
8. Cria pedido com status "pendente"
9. Redireciona para Stripe Checkout
10. Cliente completa pagamento
11. Webhook de Stripe confirma pagamento
12. Atualiza status do pedido para "confirmado"
13. Envia email de confirmação
14. Atualiza estoque
```

---

8. Tipos TypeScript Principais

TypeScript

```typescript
// lib/types/index.ts

// Auth
export interface AuthSession {
  user: {
    id: string
    email: string
    name: string
    tenantId: string
    roles: string[]
    permissions: string[]
  }
  expires: string
}

// Produtos
export interface Produto {
  id: string
  name: string
  descricao?: string
  categoriaId: string
  tipo: "tinto" | "branco" | "rosé" | "espumante" | "suco" | "grappa"
  preco: number
  precoDistribuidor?: number
  estoque: number
  safra?: number
  uva?: string
  alcool?: number
  volume?: string
  imagem?: string
```

```
  ativo: boolean
  criadoEm: Date
  atualizadoEm: Date
}

// Vendas
export interface Pedido {
  id: string
  clienteId: string
  itens: ItemPedido[]
  status: "pendente" | "confirmado" | "enviado" | "entregue" | "cancelado"
  total: number
  frete?: number
  desconto?: number
  canal: "ecommerce" | "distribuidor" | "on_trade"
  criadoEm: Date
}

// Enoturismo
export interface Agendamento {
  id: string
  clienteId: string
  tipo: "visita" | "degustacao" | "evento" | "curso"
  dataAgendamento: Date
  quantidadePessoas: number
  status: "confirmado" | "cancelado" | "realizado"
  preco: number
}

// Equipe
export interface Funcionario {
  id: string
  nome: string
  email: string
  cargo: string
  departamento: "vendas" | "enoturismo" | "producao" | "administrativo"
  roles: Role[]
  status: "ativo" | "inativo"
}

// Analytics
export interface MetricaDashboard {
  receita_total: number
  receita_mes: number
  pedidos_mes: number
  visitantes_mes: number
  taxa_conversao: number
```

```
  ticket_medio: number
}
```

---

9. Hooks Customizados

TypeScript

```
// lib/hooks/useAuth.ts
// Acesso a dados de autenticação, logout

// lib/hooks/useTenant.ts
// Acesso a dados do tenant atual

// lib/hooks/usePermission.ts
// Verificação de permissões
// Exemplo: usePermission("produtos:create")

// lib/hooks/useFetch.ts
// Fetch com cache, loading, error handling

// lib/hooks/useForm.ts
// Gerenciamento de formulário com validação Zod
```

---

10. Variáveis de Ambiente

Plain Text

```
# NextAuth
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=your-secret-key

# OAuth Google
GOOGLE_CLIENT_ID=your-client-id
GOOGLE_CLIENT_SECRET=your-client-secret

# Banco Master
DATABASE_URL=postgresql://user:password@localhost:5432/vinicola_master

# Stripe
STRIPE_PUBLIC_KEY=pk_test_...
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

```
# Email
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password

# AWS S3 (para upload de imagens  )
AWS_ACCESS_KEY_ID=your-key
AWS_SECRET_ACCESS_KEY=your-secret
AWS_S3_BUCKET=vinicola-images
AWS_REGION=us-east-1
```

---

11. Dependências Principais

JSON

```json
{
  "dependencies": {
    "next": "^14.0.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "next-auth": "^5.0.0",
    "@prisma/client": "^5.0.0",
    "tailwindcss": "^4.0.0",
    "@shadcn/ui": "latest",
    "zod": "^3.22.0",
    "recharts": "^2.10.0",
    "zustand": "^4.4.0",
    "axios": "^1.6.0",
    "date-fns": "^2.30.0",
    "react-hook-form": "^7.48.0",
    "stripe": "^13.0.0",
    "nodemailer": "^6.9.0",
    "pdfkit": "^0.13.0"
  },
  "devDependencies": {
    "typescript": "^5.0.0",
    "@types/node": "^20.0.0",
    "@types/react": "^18.0.0",
    "prisma": "^5.0.0",
    "tailwindcss": "^4.0.0"
  }
}
```

---

12. Deployment e DevOps

12.1 Estratégia de Deployment

Frontend: Vercel (Next.js otimizado)**Backend:** Vercel Serverless Functions (API Routes)**Banco Master:** Railway PostgreSQL**Bancos Tenant:** Railway PostgreSQL (múltiplas instâncias)**Storage:** AWS S3 (imagens de produtos, eventos)**Email:** SendGrid ou AWS SES**Pagamentos:** Stripe

12.2 CI/CD Pipeline

YAML

```yaml
# .github/workflows/deploy.yml
- Lint (ESLint, Prettier)
- Type Check (TypeScript)
- Tests (Jest, Vitest)
- Build (Next.js build)
- Deploy to Vercel
- Database Migrations (Prisma)
```

12.3 Monitoramento

- Sentry para error tracking
- LogRocket para session replay
- Vercel Analytics para performance
- Datadog para infrastructure monitoring

---

13. Segurança

13.1 Medidas de Segurança

Autenticação:

- JWT com expiração
- Refresh tokens
- 2FA opcional

Autorização:

- RBAC (Role-Based Access Control)

- Verificação de tenant em todas as rotas
- Rate limiting

Dados:

- Criptografia em trânsito (HTTPS)
- Criptografia em repouso (senhas com bcrypt)
- Isolamento de banco por tenant

API:

- CORS configurado
- CSRF protection
- Input validation (Zod)
- SQL injection prevention (Prisma)

---

14. Roadmap de Desenvolvimento

Fase 1: MVP (Semanas 1-4)

- [ ] Setup inicial (Next.js, Prisma, NextAuth)
- [ ] Autenticação (email/senha + Google)
- [ ] Multi-tenancy (banco isolado)
- [ ] Módulo de Produtos (CRUD básico)
- [ ] Dashboard simples

Fase 2: Core Features (Semanas 5-8)

- [ ] Módulo de Vendas (pedidos, clientes)
- [ ] Módulo de Enoturismo (agendamentos)
- [ ] Módulo de Equipe (funcionários, papéis)
- [ ] Gestão de permissões

Fase 3: Advanced Features (Semanas 9-12)

- [ ] Módulo de Analytics (dashboards, gráficos)
- [ ] Integração com Stripe

- [ ] Clube de assinatura
- [ ] Relatórios e exportação

Fase 4: Polish & Launch (Semanas 13-16)

- [ ] Testes (unitários, integração)
- [ ] Performance optimization
- [ ] Security audit
- [ ] Launch público

---

15. Conclusão

Esta arquitetura fornece base sólida para um **SaaS de gestão para vinícolas**, combinando:

- **Segurança:** Multi-tenancy, autenticação robusta, RBAC
- **Escalabilidade:** Arquitetura serverless, banco isolado por tenant
- **Usabilidade:** Interface intuitiva, dashboards personalizados
- **Funcionalidade:** Cinco módulos completos cobrindo todo o negócio

O sistema está pronto para desenvolvimento iterativo, começando pelo MVP e evoluindo para features avançadas conforme necessidade.

---

16. Referências e Recursos

- Next.js Documentation
- NextAuth.js Documentation
- Prisma Documentation
- Tailwind CSS Documentation
- shadcn/ui Components
- Multi-Tenancy Patterns
- RBAC Implementation